

Branching and Deployment Approach for Terraform Infrastructure Repositories

1. Current Setup

We currently use the PECA-IAC repository for infrastructure deployments. Terraform configuration files are maintained in the repository root and are shared across all environments.

The environments currently available are:

- Dev
- Test
- Stage
- Prod

Each environment maintains its own dedicated Terraform state file.

Infrastructure deployments are executed through reusable workflows from the ODI workflow template repository. Environment selection is based on the current branching strategy:

Branch	Environment
Main	Prod
Release	Stage
Develop	Test
Feature Branches	Dev

Currently, the Develop branch is the source branch for ongoing development activities. Developers create feature branches from Develop and perform deployments into the Dev environment.

2. Current Challenges

The major challenge exists in the Dev environment.

Since all feature branches deploy to the same Dev environment and share the same Dev state file, multiple developers working simultaneously can impact each other's testing.

A common scenario:

1. Developer A deploys infrastructure changes to Dev.
2. Developer A starts functional testing.
3. Before testing is completed, Developer B deploys another change to Dev.
4. Terraform updates the shared state and infrastructure.
5. Developer A's deployed resources may be modified or removed depending on the change.
6. Developer A must redeploy and repeat testing.

As the number of developers increases, this results in:

- Increased waiting time
- Frequent redeployments
- Interrupted testing activities
- Reduced productivity
- Higher coordination overhead

Since the state file approach is required and cannot be changed, an alternative approach is needed to reduce these conflicts.

3. Proposed Interim Solution

Note: This is a temporary approach to address the current challenges. The long-term direction remains Trunk-Based Development, but before moving to that model we should first standardize deployments, reduce conflicts, improve code quality, and establish consistent deployment practices across environments.

i. Pre-Develop Branch

As a temporary solution, introduce a dedicated branch named:

pre-develop

This branch will be created from the current Develop branch and will act as the deployment branch for the Dev environment.

Proposed Flow

```
Main
|
Develop
|
Pre-Develop
|
Feature Branches
```

How It Works

- Create a new branch called Pre-Develop from the current Develop branch.
- Existing developers who are already working on feature branches should merge their completed and reviewed changes into Pre-Develop.
- Any new development work intended for Dev deployment should create a feature branch from Pre-Develop instead of Develop.
- Since Pre-Develop contains the latest integrated Dev changes, every developer starts with the most recent code base.
- Once changes are approved and merged into Pre-Develop, deployments are performed to the Dev environment.
- Functional testing can continue while preserving previously deployed changes.
- This approach allows multiple developers to contribute to the same Dev environment without repeatedly overwriting each other's work.

ii. Team Responsibilities

Whenever a merge occurs into Pre-Develop:

- All developers should be notified.
- Developers should pull the latest changes.
- Local branches should be rebased or synchronized.
- Testing should always use the latest version available in Pre-Develop.
- Any deployment should be performed only after taking the latest changes from Pre-Develop.

This helps avoid unexpected deployment results caused by outdated local code.

iii. Merge Conflict Handling

Merge conflicts are expected when multiple developers modify the same Terraform files.

Examples:

- Same resource block
- Same variable definition
- Same module configuration
- Same output block

When conflicts occur:

- Developers should review the changes carefully.
- Resolve only the conflicting sections.
- Preserve intended functionality.
- Re-test before merging.

Merge conflicts are normal and should be resolved rather than avoided.

iv. Pull Request and Review Process

The existing Pull Request process should continue.

Regardless of:

- Branch
- Environment
- Resource Type

The following should always happen:

- Pull Request creation
- Peer review
- Validation checks
- Terraform plan review
- Approval before merge

This helps reduce deployment risks and ensures infrastructure quality.

v. Resource Control Through Terraform Meta Arguments

To support multiple environments and future promotion activities, infrastructure should be controlled through Terraform variables and meta arguments.

This allows the same Terraform code to be reused while controlling resource deployment behavior using configuration values.

Example 1 – Create Resource Only for Production

```
resource "azurerm_storage_account" "prod_storage" {  
  count = var.environment == "prod" ? 1 : 0  
  
  name          = "prodstorage001"  
  resource_group_name = var.rg_name  
  location      = var.location  
}
```

Result:

- Prod → Resource created
- Other environments → Resource not created

Example 2 – Optional Resource Creation

```
resource "azurerm_key_vault" "kv" {  
  count = var.enable_keyvault ? 1 : 0
```

```

name          = var.kv_name
location      = var.location
resource_group_name = var.rg_name
}

```

Result:

- True → Resource created
- False → Resource skipped

Example 3 – Create Resources Only When List Exists

```

resource "azurerm_resource_group" "rg" {
  for_each = length(var.resource_groups) > 0 ? var.resource_groups : {}

  name     = each.key
  location = each.value.location
}

```

Result:

- Values provided → Resources created
- Empty collection → Nothing created

Example 4 – Deploy Resources Based on Environment

```

resource "azurerm_storage_account" "storage" {
  for_each = var.environment == "prod" ? var.storage_accounts : {}

  name           = each.value.name
  resource_group_name = each.value.rg
  location       = each.value.location
}

```

Result:

- Prod → Storage accounts created
- Other environments → No deployment

Using conditions helps ensure only the required resources are deployed in a specific environment and prevents unnecessary changes during promotions.

vi. Importance of Parameterization

Infrastructure differences between environments should be controlled through variables and

tfvars files rather than separate Terraform implementations.

Examples:

- Resource Names
- SKUs
- Tags
- Network Ranges
- Monitoring Settings
- Security Settings

Benefits:

- Easier promotions
- Consistent deployments
- Less duplicate code
- Reduced manual changes
- Better maintainability
- Reduced deployment conflicts
- Better control over environment-specific configurations

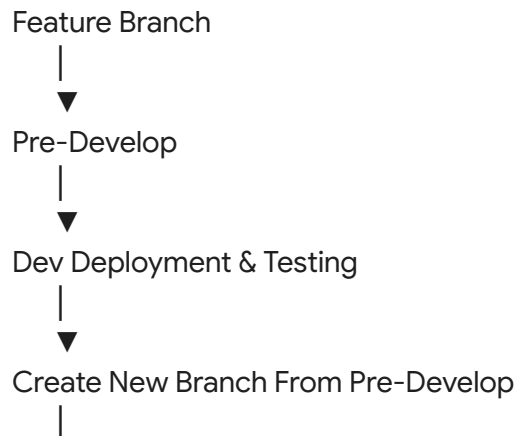
When moving from Dev to Test or Stage to Prod, the same Terraform code can be reused with different values.

This helps ensure infrastructure consistency while allowing environment-specific customization where required.

4. Promotion of Changes Across Multiple Environments

Once a developer completes validation in Dev and the changes are already merged into Pre-Develop, the same code base can be promoted to higher environments.

Example Flow





Test Deployment & Testing

If a developer is ready to continue testing in the Test environment:

- The developer should create a fresh branch from Pre-Develop.
- Since the latest Terraform configuration already exists in Pre-Develop, the developer can continue from the latest integrated code.
- Environment-specific behavior should be controlled using variables, tfvars files, and meta arguments.
- New Terraform changes should not impact existing Test resources unintentionally.
- Resource behavior differences between environments should always be controlled through conditions and variables.

The objective is to ensure that the same Terraform code can be promoted safely while allowing environment-specific configurations where required.

Gradual Elimination of Long-Lived Branches

This approach is intended as a transition strategy.

Once approximately 90% of Dev environment activities become stable:

- Pre-Develop can gradually be eliminated.
- Dev and Test deployments can start directly from Develop.

Similarly, once Test activities become stable and standardized:

- Environment-specific branches can be reduced further.
- Promotion can move closer to Main.

Over time the branch structure becomes simpler:

- **Current:** Feature → Pre-Develop → Develop → Release → Main
- **Future:** Feature → Develop → Main
- **Final:** Feature → Main

This gradual transition allows teams to adopt trunk-based practices without disrupting current delivery activities.

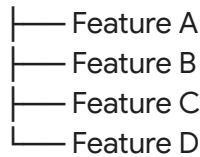
5. Long-Term Direction

i. Trunk-Based Deployment

The long-term goal should be moving towards a Trunk-Based Development model.

Target State

Main



- Main becomes the single source of truth.
- Developers create short-lived feature branches directly from Main.
- Changes are:
 - Reviewed
 - Approved
 - Merged quickly
- Deployments are performed from Main across environments.

Benefits include:

- Faster integration
- Reduced branch management
- Fewer merge conflicts
- Better visibility
- Easier maintenance

ii. Leverage Terraform Reusable Modules

As infrastructure grows, reusable modules become increasingly important.

A module allows infrastructure logic to be written once and reused multiple times.

Key benefits include:

- **Build Once, Reuse Many Times:** The same module can be consumed across multiple environments and resource deployments by passing different inputs.
- **Reduced Duplicate Code:** Common infrastructure patterns are implemented once rather than repeatedly copied.
- **Easier Maintenance:** A change made in a module can be versioned and reused consistently.
- **Consistency Across Environments:** All environments use the same implementation pattern, reducing configuration drift.
- **Controlled Upgrades:** Module versions can be tagged and consumed when teams are ready, avoiding unexpected impacts.
- **Faster Onboarding:** New infrastructure deployments can consume existing modules instead of building from scratch.
- **Centralized Management:** Common infrastructure logic can be maintained in one place and shared across multiple deployments.

iii. Challenges with Long-Lived Branches

Long-lived environment branches and development branches introduce several operational challenges.

Frequent Merge Conflicts

The longer a branch remains active, the more difficult merges become.

Developers often modify the same files over time, resulting in larger and more complex conflicts.

Environment Drift

Different branches may contain different infrastructure definitions.

Over time:

- Dev differs from Test
- Test differs from Stage
- Stage differs from Prod

This makes deployments less predictable and harder to troubleshoot.

Delayed Integration

Changes remain isolated for long periods.

Issues are discovered later rather than earlier.

Higher Maintenance Effort

Teams spend more time:

- Syncing branches
- Resolving conflicts
- Performing rework

instead of delivering infrastructure changes.

Summary

The proposed approach introduces a temporary Pre-Develop branch to reduce Dev environment deployment conflicts while preserving the existing Terraform state file strategy.

The solution focuses on:

- Controlled Dev deployments
- Pull request reviews
- Peer validation
- Variable-driven configurations
- Terraform meta arguments
- Consistent promotion processes

- Gradual branch simplification

As environments stabilize, long-lived branches can be reduced step by step until the organization reaches a trunk-based deployment model where Main becomes the single source of truth.

This approach balances current operational needs while moving towards a simpler, more scalable, and maintainable Terraform delivery model.